

Design for a Tape Spring Five-Bar Parallel Robot Arm System Incorporating Edge AI Computer Vision

Dave Riley (52207153)

MEng Electrical & Electronic Engineering with Robotics

April 27, 2026

Abstract

In this paper, the design of a system to autonomously control a 2-degree-of-freedom five-bar robotic arm formed from a single tape spring is described and evaluated. The initial prototype produced by this project is inexpensive, lightweight and folds to a compact size. Challenges, including motor resonance and joint tension, were identified and the effectiveness of countermeasures was evaluated. For the vision part of the system, a YOLO (You Only Look Once) CNN (convolutional neural network) was trained and deployed on an Nvidia Jetson Orin Nano for object detection and classification. The CNN detected objects correctly 99.6% of the time and classified them correctly 92.2% of the time while running at a mean rate of 36 frames per second. Testing indicated that the final system design presented in this paper performed adequately for lower precision tasks, with a mean system error of 19.5 mm. A mechanism to allow the arm to autonomously change its bar length, which is currently being developed, would improve this. In addition, areas for further research and key component upgrades were identified, which would significantly increase the arm's performance for a modest hardware cost increase.

1 Introduction

The overall trend within industry towards automation of complex tasks using smart machines, smart facilities and similar concepts which exchange information autonomously is often considered a key component of “industry 4.0”, or the fourth industrial revolution [18]. Some authors consider it an extension of industry 3.0, which started in the 1970s and refers to the automation of jobs that were previously done manually. Across both developing and developed countries, increasing efficiency by leveraging aspects of the industry 3.0 and 4.0 concepts are a key goal of many organisations [8].

Robotic arm systems are a common type of industrial robot used to augment human workers or replace them entirely in tasks that involve holding, lifting, or moving objects or tools [6]. Arm technology continues to advance at a rapid pace, largely driven by industry demand [19]. The most common type of robotic arms are serial manipulators, that is, arms where each bar is connected one to the next by motor actuated joints running from the base to the end effector. Parallel manipulators have more than one serial chain of bars connecting the end effector to the base, and are often designed so that the passive joints do not require motors. Parallel manipulators, particularly five-bar planar arms, are widely used for automating tasks in agriculture (for example, vegetable planting), logistics (for example, picking and packing), and in precision manufacturing [11].

Tape springs are thin, straight sections of material with transverse curvature, which greatly increases their stiffness, for example, the tape of a self retracting tape measure. Tape springs are often made of

steel or composite materials, with the latter being extensively used in tape spring hinges, which are used as deployment mechanisms for structures in space satellites [32]. Tape springs are easily spooled for compact storage, and when made from steel, they are relatively low-cost.

Conventions used in this paper are that for arm descriptions, R means a revolute joint, with an underline meaning the joint is actuated, and the preceding number indicating the number of bars, for example, in 2-DOF (Degrees Of Freedom) 5-RRRRR arm. Arm mode is used to describe the combination of the mode of the left and the right non-actuated joints on this type of arm, with a joint mode meaning if the joint is greater or less than 180° when measured counterclockwise from the distal link.

The type of robot arm presented in this paper is a 2-DOF (Degrees Of Freedom) 5-RRRRR planar parallel manipulator. The whole of the arm (4 of the 5 bars, with the 5th bar being the fixed base between the motors) is formed from a single tape spring in a bistable configuration and is positioned by the two motors, as shown in figure 1(a). The arm's central joint node is used to attach an end effector, and only the two joint nodes fixed to the base have motors, enabling the arm to be lightweight and therefore able to accelerate quickly and with less energy than a serial manipulator design of equivalent stiffness.

Three joint nodes are able to slide along the tape spring. They flatten the transverse curvature of the tape spring at their centre, significantly reducing the stiffness of the tape spring at that point and forming a revolute joint. This design enables the arm to be folded away, deployed, and the length between joints adjusted, giving it characteristics that are quite different to most other five-bar closed-chain robotic arms. Similar 5-RRRRR arms often have joints with low friction and that are free to move 360° [5]; conversely, this arm has tension in the joints which resists bending, and the joints have a limited range of motion due to the joint node geometry. The use of a tape spring as opposed to rigid materials for the arm presents design challenges, including limited stiffness, fatigue, vibrations and resonance.

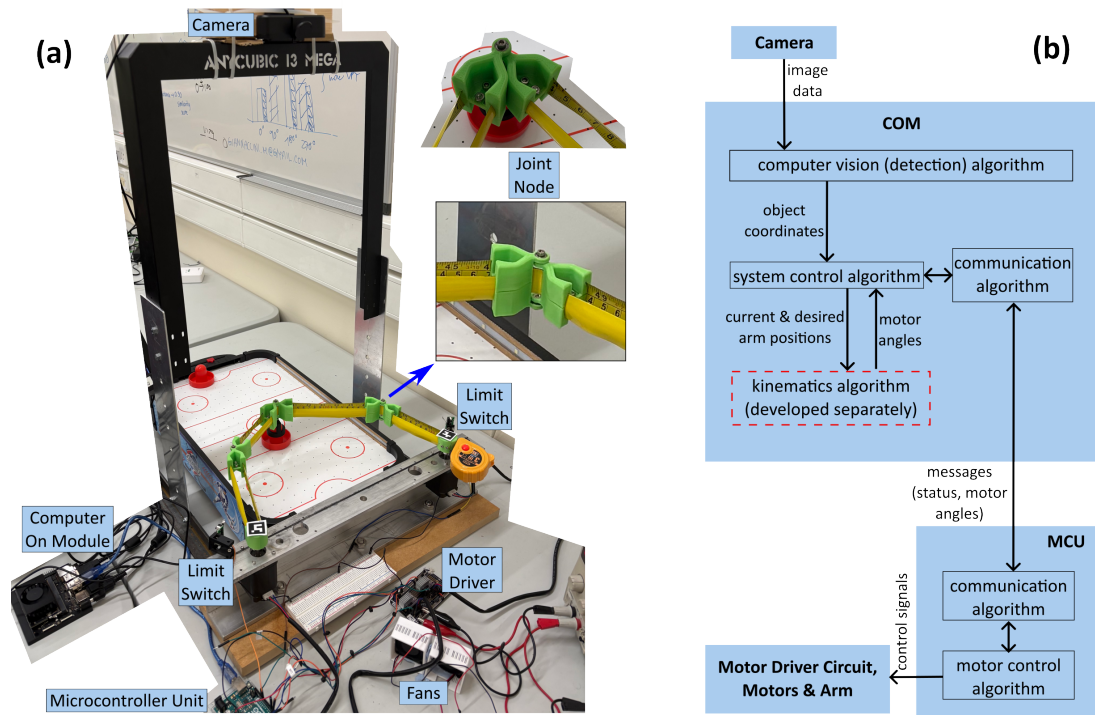


Figure 1: System overview. (a) Prototype system during testing. (b) System architecture overview, showing key algorithms on the COM (Computer On Module) and MCU (Microcontroller Unit).

Potential benefits of the arm presented in this paper include low manufacturing costs, due to its simple construction, and low storage and transport costs, due to its low weight and small size when folded. With further development work, it is intended that it will be able to autonomously adjust the length between

its joints, and so optimise itself for particular movements mid-task.

The intended uses for production versions of the arm include as a “third arm”, worn by a person on a torso-mounted harness and able to autonomously carry out tasks such as picking and placing items in a warehouse or to assist a person who has a physical impairment in their daily life. It may also find use on small robots that need to manoeuvre in confined spaces, which need some sort of lightweight arm capability, such as robots used in hazardous area operations. Due to low manufacturing costs, it may also be suited to applications where numerous arms are controlled by one computer vision system.

The aim of this project is to develop a prototype arm system that uses computer vision to detect objects and moves the arm to accomplish a task based on this information. The system will need to interface with a kinematics algorithm, which is being developed by a parallel project. It will be an early prototype, and so needs to be suitable for experimentation and iteration by multiple researchers whilst maximising the positive characteristics of the arm and overcoming its challenging characteristics. A key focus of this project will be to produce an inexpensive system. The motivation for this is that cheap manufacturing costs are one of the key benefits of the arm design, so starting with a prototype that builds on this strength extensively will help inform future research and design decisions. If the design started with, for example, the use of expensive high-end servo motors, then it would be difficult to quantify if they could be downgraded to a cheaper motor type, however by starting with the cheaper option and quantifying their performance an informed decision can then be made regarding upgrades.

This paper reviews the current state of the literature for similar robotic arm systems (section 1.1), describes the methodology and proposes a system design (section 2), reports and discusses results from testing (section 3), then presents a conclusion (section 4). Supplementary materials, including relevant source code, risk assessment for lab work and test data are provided alongside this paper and are referenced with an S prefix, for example “see code in S1.1”.

1.1 State of the Art

During the literature review for this project, no papers were identified which explored a design of the type used in this project; however a number of papers were found to contain concepts that had some similarities or other relevant information. Campos et al. [5] developed an industrial prototype for a 2-DOF 5-RRRRR planar parallel robot arm optimised for pick and place operations. To maximise the arms workspace they made all bars identical in length, which increases parallel singularities, then developed methods to switch the arms work mode to overcome the parallel singularities, resulting in an overall increased workspace when compared to other 2-DOF 5-RRRRR arms with an equal total sum of the bar lengths. Their prototype featured continuously rotatable low friction passive joints and utilised relatively high-end direct drive servomotors with encoders to obtain the high torque (peak $51 \text{ N} \cdot \text{m}$, continuous $17 \text{ N} \cdot \text{m}$) and low backlash required to meet the objectives of their arm. Joubair et al. [17] built on this work by developing a low-cost calibration procedure and demonstrating that the prototype developed by Campos et al. [5] could achieve errors of 0.08 mm or less for the arm’s entire workspace of $600 \text{ mm} \times 600 \text{ mm}$, making the design suitable for precise work. This project aims to produce a much lower cost and lower performance system than this, however the ability to change working mode may prove useful in accessing a larger proportion of the workspace given that the joints will have a limited range of motion.

Shinjo et al. [27] developed a 5-bar 2-DOF robotic arm optimised to play miniature air-hockey for use in human-robot interaction studies. The authors provide several points of comparison for benchmarking. They state that other robotic arms used for playing air-hockey cost between \$7000 and \$135000, whereas the cost of theirs (prices adjusted to the present day) is around \$380, excluding the cost of a computer to run the system and the cost of an air hockey table. The two actuated joints share an axis of rotation, simplifying the kinematics, and the arm is very lightweight with low-friction joints, allowing the use of relatively cost-effective (approximately \$50 each) low-torque (peak $0.5 \text{ N} \cdot \text{m}$) servo motors. The computer vision camera (model USBFHD01M-SFV ELP) was used at 30 frames per second. The

resolution used is not stated, but the maximum for that camera model and frame rate is 1920x1080p. The camera cost approximately \$100 and sent images to an algorithm that detected the puck using the HSV colourspace. The motor position was updated at 10 Hz, and the system was able to effectively return the puck when playing against a human. A similar camera setup may be suitable in this project, however more expensive servo motors with increased torque would be required to handle the increased resistance presented by the arm in this project, so alternate cheaper types of motor may need to be explored.

He et al. [14] used laminated bistable tape springs to create deployable “fingers” for an effector. The resulting prototype folded away and had high compliance in a manner similar to soft robots, allowing it to safely interact with unknown or complex objects. Ding et al. [10] made use of similar tape springs as the main extending members supporting a planar manipulator able to extend to 1.9m length from a collapsed length of 0.18m, showing how effective the use of bistable tape springs can be. They found that joining two C-shaped-section tape springs together into a closed shape provided significantly higher stiffness than a single C-shaped-section tape spring alone, yet still had very low stiffness when flattened, and so was easily coiled or bent. These papers show that the bistable tape spring arrangement can be very robust; however, they both laminated the tape springs together, whereas the arm in this project cannot accommodate that at present. If issues with rigidity are encountered during testing, then consideration could be given to modifying the arm to use laminated tape springs.

Mallikarachchi and Pellegrino [20] modelled the behaviour of tape spring hinges made from composite material, focusing on their storing and releasing of strain energy when being folded and then self-deploying. This storing and releasing of energy is not dissimilar to how the arm presented in this paper behaves when the joints bend. Osele et al. [23] used a principle similar to the one used in the joint nodes on the arm described in this paper. Their manipulator pinched a pair of bistable tape springs to create a single revolute joint for use in an aerial robot. Because their design is not closed-chain, the tension caused by the joint is not a challenge in the way that it is for the arm presented in this paper.

2 Methodology

2.1 Proposed Solution

2.1.1 System Architecture

Figure 1(b) shows a simplified overview of the system architecture. A modular approach was taken to hardware and software architecture, with defined interfaces to allow easy changing of components where practicable, enabling quicker prototyping, testing and design iteration.

The Computer On Module (COM) can be any device with an appropriate operating system, sufficient computing power to run the main algorithms, and connectivity to interface with the camera and the MCU. Many single-board computers are suitable, and a laptop was sometimes used during development. The Microcontroller Unit (MCU) controls the motor driver circuit, sending precisely timed control signals and tracking the position of the motor shafts. By integrating a sufficiently powerful COM, the system can operate autonomously, with minimum lag and with increased security. If maximum security was desired, then the system can be operated air gapped (with no connection to a data network). This independence also makes the system easier and cheaper to deploy and integrate for end users.

2.1.2 Motor Driver Circuit and Motors

The key requirements for the motors are that their shafts can move to requested positions with sufficient repeatability and accuracy, and that they have sufficient torque both to move the arm and to hold the arm in place when it is not moving. Arms of a 5-RRRRR configuration often use servomotors [11] [5]. The characteristics of hybrid permanent-magnet stepper motors mean that they may also be suitable in this application [15, Chapter 10]. Table 1 compares the characteristics of typical stepper and servo motors

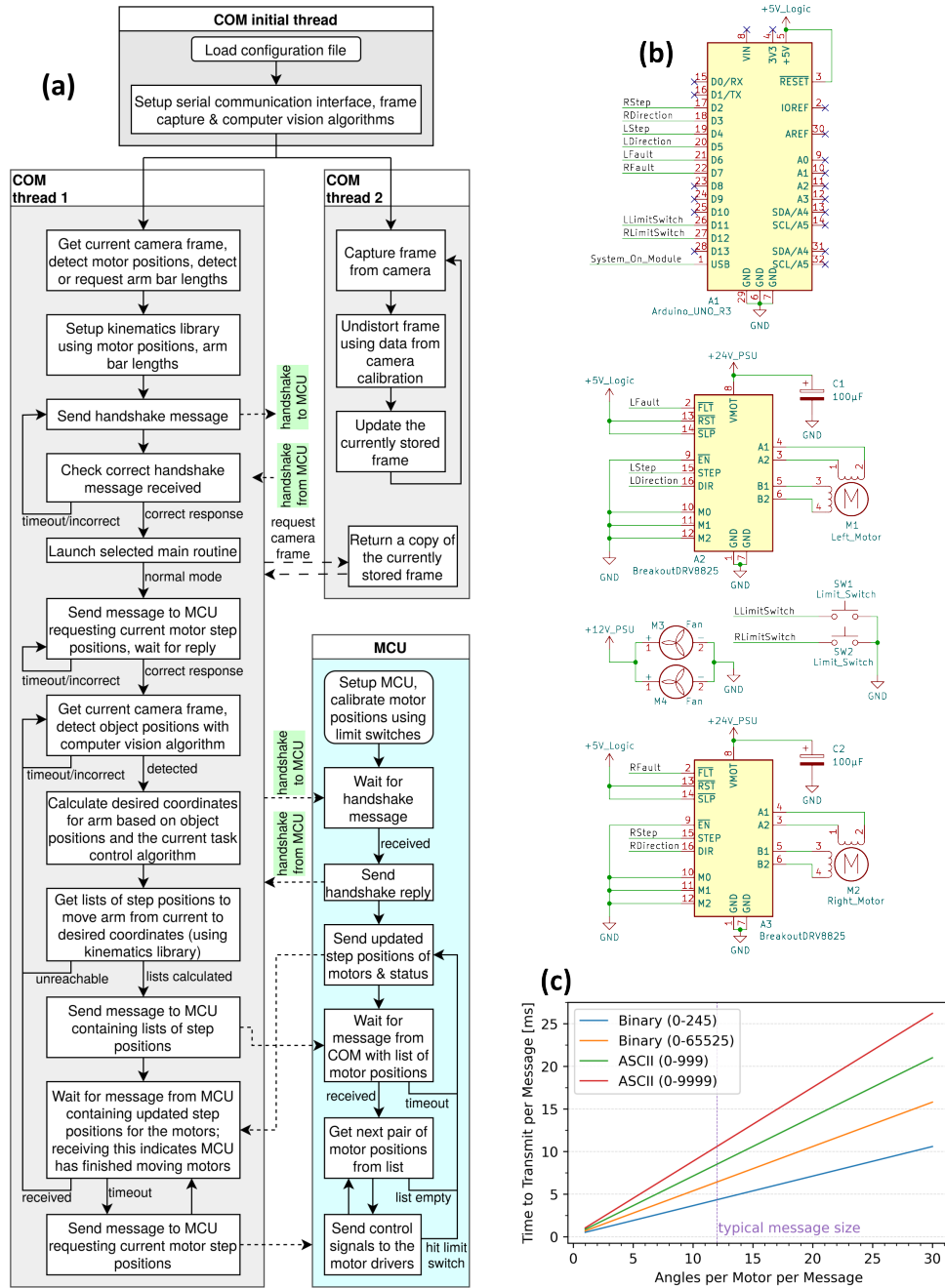


Figure 2: System design. (a) Overview of the algorithms running on the COM and MCU. (b) Electrical schematic showing the MCU, motor driver circuit and limit switches. (c) Estimated times for the slower Uno R3 to receive angles messages over different protocols with different maximum angle sizes.

that are important to this project. As can be seen, a suitable servo motor would generally perform better, however a stepper motor in an open-loop configuration could be suitable for a lower precision and lower speed arm, enabling it to be produced at a much lower cost. This prototype system uses stepper motors to explore the viability and performance potential of this cheap-to-produce configuration.

$$i(t) = \frac{U}{R} \left(1 - e^{-\frac{R}{L}t}\right) \quad (1)$$

Stepper motors are brushless DC motors which turn in discrete steps. This design uses hybrid permanent magnet stepper motors, which use both permanent magnets and magnetic reluctance to generate a given level of torque in a smaller motor size than other stepper motor types. The discrete steps are

Table 1: Key differences between servo and stepper motors in the context of this project.

	Stepper Motor		Servo Motor
Torque profile	T_{hold} when static, decreases as speed increases		T_{rated} or greater below rated speed
Motion	discrete steps		continuous
Power efficiency	poor		good
Control type	open-loop	closed-loop (with encoder)	closed-loop
Cost	lowest	medium	highest
Position error accumulates?	yes	no	no

produced by exciting the stator phases in a sequence, with maximum torque achieved when the phase current reaches its rated value. Due to resistance and inductance, the current takes time to reach its rated value, as can be seen by equation 1, the equation for a series LR inductor circuit where $i(t)$ is the current at time t , U is the potential difference across the circuit, R is the resistance, and L is the inductance. At higher speeds, motor torque reduces due to there being insufficient time for the current to reach its rated value. Torque can be maintained to higher speeds by supplying a voltage much higher than the rated voltage and using a control circuit to limit the current to its rated value, resulting in a faster rise time for the current. This is the approach taken for the prototype, facilitated by a Texas Instruments DRV8895 IC which also provides protection against various fault conditions [29].

Different methods of exciting stepper motor stator phases produce step sizes that range from full steps down to (for the DRV8825) $\frac{1}{32}$ of a step. Smaller step sizes result in smoother movement, but torque at the intermediate steps is reduced. In full-step 2-phase mode the stator always has two phases excited at a time, and the motor has maximum torque. In half-step mode, the excitement routine alternates between two phases on and one phase on, doubling the number of positions that the stator magnetic field is able to move the rotor to at the expense of torque being approximately 71% of maximum when in positions where only a single phase is excited. As step size becomes smaller, with the current being shifted between phases in increments that approximate a sinusoid wave to achieve these smaller microsteps, the lowest torque level in a microstep position continues to decrease rapidly, with $\frac{1}{4}$ having approximately 38% of T_{hold} and $\frac{1}{16}$ having 10%. This can result in the motor moving several microsteps at once or losing steps if the load torque exceeds the lower motor torque in these microstep positions. Microstepping helps reduce vibration and resonance issues, and so is employed for this prototype ($\frac{1}{16}$ steps are used), but the arm only pauses in full step positions (see S2.1 motorController.ino). This means that the motion will be smoother than if full stepping was used, but if the torque of microstep positions is insufficient it will not result in step loss as long as the torque in full step positions is still sufficient.

$$f_n = \frac{1}{2\pi} \sqrt{\frac{T'}{J_M + J_L}} \quad (2)$$

$$T_{pos} = -T_{hold} \sin \theta \Rightarrow \frac{dT_{pos}}{d\theta} = T_{hold} \cos \theta \quad (3)$$

$$T' = \frac{T_{hold}}{\frac{\pi}{180} \frac{360}{N}} = \frac{T_{hold}}{\frac{2\pi}{N}} = \frac{NT_{hold}}{2\pi} \quad (4)$$

$$I_O = \frac{ML^2}{3} \quad (5)$$

Stepper motors can be modelled as underdamped second order systems [3], and experience issues with resonance when stepped at certain frequencies and with certain load and operating conditions. Equation 2, where T' is the stiffness of the torque-position characteristic, and J_M and J_L are the mass moment

of inertia of the motor and load respectively, can be used to estimate the natural frequency of the motor or system [1]. Step rates which are close to this natural frequency or its sub-harmonics are liable to result in excessive vibration and skipped steps due to resonance, although damping and the nature of the load can significantly change both the resonance frequency and response characteristics [2].

Equation 3 shows how the static torque-position characteristic (T_{pos}), which is the amount of torque the motor develops to counter load torque for a given rotor displacement angle (θ) from the equilibrium position, can be approximated when there is no torque-position characteristic curve available by assuming that the characteristic curve is sinusoidal [1]. From this, the slope of the curve at the equilibrium can be approximated, and the stiffness of the torque-position characteristic (in units of $\text{N} \cdot \text{m} \cdot \text{rad}^{-1}$) can be calculated as shown in equation 4, where N is the number of steps per revolution. This was used to estimate natural frequencies for the motors as 225 Hz for motor A, 232 Hz for motor B and 229 Hz for motor C. The natural frequency for motor C with the arm attached was estimated by approximating half of the arm as a uniform rod attached to a motor and calculating the load inertia using equation 5. The total mass of the arm was 0.218 g, giving natural frequency estimates of between 12 Hz and 14 Hz.

Initial testing indicated that the torque required to move and hold the arm varied depending upon the joint angles, the overall position of the arm, the bar lengths and any discrepancies in the length of the tape springs on each side of a bar which sometimes resulted from the arms movement history. Several stepper motors were available to borrow for testing, so an experimental approach was taken to motor selection. Figure 2(b) shows the connections for the motor driver circuit, including the limit switches, the fans to cool the motor drivers, and the MCU.

Using the computer vision system to provide feedback for closed-loop motor control was briefly considered, but for typical applications, this would likely result in a combination of inadequate accuracy and excessive processing time, so the idea was not explored further. This prototype uses open-loop stepper motors to minimise cost. For deployment in applications with long run times, such as a continuous industrial application, the open-loop system could use a software routine where the arm position calibration routine described in section 2.1.7 is repeated every certain number of movements or period of time as an additional mitigation against step loss.

Table 2: Key characteristics of the motors that were tested.

	Motor A	Motor B	Motor C
Datasheet reference	[25]	[26]	[22]
Body size wxdxh [mm]	42 × 42 × 33	42 × 42 × 38	42 × 42 × 60
Mass [g]	220	280	578
Holding torque [$\text{N} \cdot \text{m}$]	0.22	0.36	0.80
Max. current per phase [A]	1.33	1.68	2.00
Rotor inertia $\text{kg} \cdot \text{m}^2$	3.5×10^{-6}	5.4×10^{-6}	12.3×10^{-6} *
Common to all	bipolar hybrid permanent-magnet stepper motor (4 wires), NEMA-17 faceplate, 1.8° full-step angle, 5% full-step accuracy		

*not provided on datasheet, estimated using datasheets from similar motors

2.1.3 Computer Vision

An Emeet SmartCam C60E 4k webcam was mounted on an overhead gantry and connected to the COM via USB, giving monocular vision. Its native resolution is 3840×2160 pixels. It was configured to a resolution of 1920×1080 to lower computational overhead. Initial testing determined that resolutions which did not divide the native resolution by 2^n , where n is an integer, resulted in reduced accuracy of computer vision measurements, possibly due to the pixel binning strategy employed by the camera.

Camera distortion was corrected by taking many (up to 100) images of a chessboard pattern in dif-

ferent orientations and locations, then using these to deduce the intrinsic properties of the camera and its distortion coefficients (S1.1 calibratecamera.py). Once camera images were able to have their distortion corrected, a second calibration routine was run where a test image was used to measure the number of pixels per millimetre at a given, or at multiple given, heights. This information was stored and used to measure distances on frames from the video feed.

Object detection algorithms, including YOLO (You Only Look Once) and fiducial markers, were explored. YOLO is an approach where the image is resized if needed, then input directly to a convolutional neural network (CNN), which simultaneously predicts bounding boxes and probabilities for the classes it has been trained on [24]. This is in contrast to many other AI vision algorithms, which often run a classifier algorithm multiple times on different areas and/or scales of the image to perform detection and classification. Because of its single pass approach, YOLO is often less computationally intensive than other AI detection algorithms, and so is better suited to deployment on edge computing devices [28].

A YOLOv4-tiny CNN [4], using the Darknet v5 C++ framework and the CSPDarknet53 backbone architecture, was trained from scratch to detect four objects (a blue block and plate, and a toy cake and kiwi). This implementation of YOLO was selected because it is available under an open source licence that allows for commercial use [7] and has been used for real-time video detection running on lower performance GPUs [21]. Other YOLO implementations were examined, most of which had higher YOLO “version numbers”, the majority of which were from a company called Ultralytics. These Ultralytics models are often considered more straightforward to train and deploy, however for commercial use, an enterprise licence is required [31] with costs reportedly starting at \$5000 per year for a small-scale project with a single developer [30], which is far too costly for this project.

Table 3: Hyperparameters used in training the YOLOv4-tiny CNN.

Hyperparameter	Value	Hyperparameter	Value	Hyperparameters Set True
Network width	480	Learning rate	2.61×10^{-3}	Recalculate anchors
Network height	256	Batch size / subdiv	64 / 1	Random crop/zoom

$$AP = \int_0^1 p(r) dr \quad (6)$$

Larger YOLO input network resolutions can detect smaller objects better, but require more memory to train and run, and take longer to complete inference. Darknet requires input width and height to be a multiple of 32, and after the image is scaled to the network size, the smallest object must be 16×16 pixels or larger for reliable detection. Input network resolution was calculated following this principle, and the other hyperparameters were selected based on guidance documents provided by the Darknet project. Table 3 shows the hyperparameters used for training. The training data was created by annotating 515 images extracted from a video of the objects being moved around, with 24% of images being negative samples and the rest having one or multiple of the objects in, sometimes partially obscured. This data was augmented using appropriate flip, rotate and zoom operations. Training was done for 12000 iterations, taking 85 minutes using an Nvidia A100 GPU, resulting in a model that had a mean average precision (mAP) of 99.67% on the initial dataset. Precision (p) is how many positives predicted by the model were correct, and recall (r) is how many of the truly positive samples were correctly predicted as positive by the model. AP (average precision) for a class is calculated as the area under the precision-recall curve for that class, as shown in equation 6. mAP is the mean of the AP across the different classes.

Aruco fiducial markers were used to mark the motor locations, and were also used to mark the location of each joint to allow measurement of bar lengths. They were selected because of their inbuilt error correction, and the algorithms available that generate a minimum-sized dictionary of markers for the desired application. The combination of the minimum-sized dictionary, along with an optimised detection algorithm, allows for relatively quick and robust detection of the markers [12].

2.1.4 Microcontroller

The MCU communicates with the COM via USB and moves the motors according to the instructions received. It interfaces with the motor driver ICs, sending correctly timed pulses to move the motors to the desired position. Aside from these requirements, the MCU needed to be cost effective and be easy to work with for other future users of the prototype. The Arduino Uno MCU development board family was selected as meeting these criteria. The Arduino C++ Hardware Abstraction Layer (HAL), based on the Wiring library [16], was used when writing the MCU software so that the code is portable across the Arduino family and several other boards. Third-party libraries were not used.

2.1.5 Computer on Module

The COM selected for this prototype system was an Nvidia Jetson Orin Nano. It has a discrete GPU and is optimised for edge AI and computer vision tasks. The COM itself (excluding the development board used in this project) has a footprint approximately the size of a credit card (69.6 mm × 45 mm), with height depending upon the size of the fan unit selected for cooling. It was used with an embedded Linux operating system, and the Nvidia Jetpack SDK was used to replace key system packages such as OpenCV with modified versions optimised for the hardware. To streamline the development process, a virtual environment which matched the libraries available on the COM as closely as possible was created, and an SSH server was set up on the COM to allow code to be pushed across easily for testing.

The algorithms for the COM were primarily written in Python. The kinematics algorithm was written by a separate project in C. This project cross-compiled the kinematics code to a shared library, which was loaded and called from within the Python codebase on the COM (see S1.5 kinematicExtLib.so, S1.14 kinematicssetup.py, S1.15 operatingmodes.py).

2.1.6 Communications

The selected MCUs (R7FA4M1AB and ATmega328p) are able to communicate with the COM via USB, either directly or via a serial to USB IC connected to the UART on the MCU. A printable ASCII application layer communications protocol was designed, loosely inspired by the sentences in protocols such as NMEA 0183 [9] but with a shorter format. A counterpart binary-based protocol was also designed, which can transmit data faster at the expense of readability during testing.

	start	msg type	M1 1 st angle	angle sep	M2 1 st angle	data sep	M1 2 nd angle	angle sep	M2 2 nd angle	end								
ASCII	\$	>	1	2	0	&	1	0	6	,	1	2	1	&	1	0	7	%
Binary	0xF6	0xF7	0x78	0xF8	0x6A	0xF9	0x79	0xF8	0x6B	0xFA								

Table 4: An example message sentence in the ASCII and binary protocols, transmitting angles 120/106 then 121/107 for motors one/two respectively.

Message types to communicate include enquire, acknowledge, error codes and lists of step positions (“angles”) for the motors to turn to. Lists of angles are the longest type and the most frequently sent. Using the number of steps from a reference point to measure motor angles ensured only integers were transmitted. Table 2.1.6 shows an example angles type message in each format. As can be seen, the binary protocol was designed to closely match the ASCII one so that swapping between them in code is straightforward. Motor positions are sent in full-steps only, requiring one byte per motor angle in binary or three in ASCII.

$$B_{tot} = 2b_{frame} + b_{type} + 2nb_{angle} + nb_{angle_sep} + (n - 1)b_{data_sep} \quad (7)$$

Equation 7, where B_{tot} is the total bytes in the message, B_{frame} the bytes per framing (start/end) character, b_{type} the bytes for the message type character, n the number of angles per motor in the message, b_{angle} the number of bytes required for an angle, b_{angle_sep} the number of bytes for an angle separator and b_{data_sep} the number of bytes for a data separator, was used to calculate angle message sizes in bytes. The serial communication protocol on the slower Uno R3 MCU uses 115200-8N1 (115200 baud, 1 start bit, 8 data bits, no parity, 1 stop bit), which was used with equation 7 to estimate times to send angle messages as shown in figure 2(c), which also includes data for sending larger ranges of angles. A typical length message with ASCII (0-999) would take approximately 8.5 ms to send to the slower Uno R3 MCU, which is acceptable for the current system design. When the system is designed for production, it will likely use a single PCB with both the COM and the MCU on it. In this case, an alternative underlying communication protocol, such as SPI, could be considered instead of USB.

The algorithm to receive and check messages on the MCU was implemented using a C switch case state machine design pattern (see S2.1 motorController.ino:msgcheck). On the COM, a more abstract approach was used which leveraged the Python Pyserial library to match characters, extract the data and flush buffers where required (see S1.6, S1.8, S1.9, S1.15).

2.1.7 System Control

Figure 2(a) gives an overview of the major system algorithms running on the COM and the MCU, and how they interact when the system is running in its normal operating mode (see S1 and S2 for the full code). The software running on the COM was written to run multithreaded, which provides a robust method for fetching the most recent frame from the camera buffer and correcting its distortion. It also increases performance by splitting up the I/O bound tasks between different threads.

At system startup, the COM and the MCU go through their own setup routines. As part of its routine, the MCU calibrates the motor positions by turning the arm left until it hits the left limit switch (position for the left motor now known and tracked), and then right until it hits the right limit switch. Once the position for both motors has been calibrated, the MCU waits for the handshake message, which is sent by the COM once it is ready. On receipt of the correct response from the MCU, the COM launches its main routine. The routine retrieves the current motor positions from the MCU, then enters an infinite loop of getting a camera frame, detecting objects, deciding how to position the arm, getting lists of motor step positions from the kinematics library to move the arm, sending the lists of steps to the MCU, and then waiting to receive the new motor step positions from the MCU.

2.2 Testing Methodology

Table 5: Summary of tests to determine the repeatability of the manual measurements taken during testing, giving an indication of uncertainty for measurements presented in the results.

	Test Point 1		Test Point 2	
	x	y	x	y
Number of measurements taken	7	7	7	7
Mean [mm]	-259.43	148.86	-229.43	220.14
Standard Deviation [mm]	1.93	1.46	2.3	0.9

To evaluate the uncertainty of manual measurements taken as part of testing, repeated measurements were taken at two different points on the workspace; the details of this are shown in table 5. To test for step loss, the arm was run in precalculated movement routines (such as S1.15 operating-modes.py:side2side_auto) and stopped at known points to check if its motor angles were correct. Profiling of code execution time was done using the cProfile and yappi Python modules (see S1.6 main.py).

3 Results and Discussion

3.1 Motor Driver Circuit, Motors and Arm

Table 6: Initial motor testing configuration and results.

	Motor A	Motor B	Motor C
<i>Config.</i>	mk1 arm, full-step mode, 24 V supply		
Driver current limit [A]	1.20	1.55	1.85
<i>Results</i>			
Min. joint angle	N/D* ¹	75°	65°* ²
consistently achievable			
Step loss ($T_M < T_L$)* ³	>1 in 5	1 in 15	none
* ¹ not determined due to excessive step loss * ² limited by joint design			
* ³ estimated by running movement routine at non-resonant speeds			

Table 6 summarises the key findings of the motor tests. Motor C was the only one with sufficient torque to be able to achieve the required joint angles without losing steps. Initially during testing, the motor drivers regularly overheated after less than 5 minutes of operation. Swapping from the regular heatsinks to much larger 9 mm × 20 mm × 12 mm aluminium ones that used high quality thermally conductive tape resolved this issue fully. Another initial issue was that the first motor mounts were 3D printed PLA parts which were not sufficiently rigid, resulting in motor and arm vibrations that caused lost steps. The rigid aluminium motor mounts seen in figure 1 resolved this issue.

As initial testing progressed, the tape spring became fatigued. This affected both the stiffness of the arm in certain areas and resulted in the bars changing length over time, especially when testing was done in a configuration that caused significant vibrations. Part way through testing, the tape spring was so fatigued that the joint nodes would slide back into a position of (left bar to right bar) 172 mm, 141 mm, 187 mm and 138 mm after only one or two movements. It appeared that the uneven fatigue along the length of the tape spring caused this to be the configuration that placed minimal tension on the joint nodes. As a consequence of this, all tests presented in this report, which were done later in testing following various improvements to the system, were done using the arm in this configuration. A mk2 arm was designed with a thicker, wider and higher quality tape spring to improve stiffness and robustness. This will store and release greater amounts of strain energy than the mk1 arm. Due to the lead time on 3D printing the mk2 arm was not manufactured in time to undertake testing for this paper.

As described in section 2.1.2, stepper motors can experience resonance issues when run at certain step rates. Figure 3 shows resonant speeds determined by quantitatively measuring step loss and qualitatively rating noise and vibration for different motor configurations and speeds. As can be seen, using full steps with no load resulted in severe resonance at speeds close to those calculated as the natural frequency and sub-harmonics in section 2.1.2. Attaching the arm to the motors adds inertia, reducing the natural frequency of the system. Using full steps with the arm attached produced significant noise and vibration, with the highest peak at half of the natural frequency of the motor, however once above 45 RPM, the arm no longer skipped steps. Using $\frac{1}{16}$ microsteps significantly reduced noise and vibration, resulting in the arm moving in smooth sweeping motions with little to no vibration. In addition, the natural frequency was pushed further away from operating speeds because of the increase in pulses per second used to obtain a given motor speed.

The mk1 arm was not stiff enough to be driven above an angular velocity of 54 RPM = $5.65 \text{ rad} \cdot \text{s}^{-1}$; at angular velocities greater than this the longest motor adjacent bar (172 mm) would bend to a point that its transverse curvature was compromised, resulting in the failure of the bar until the motor was stopped and the bar was manually straightened. Work done by Guan et al. [13] found that this failure occurred at $6.1 \text{ N} \cdot \text{m}$ for the spring tape in their study; the methods they used (a combination of finite element analysis and physical experiments) could be utilised to calculate a safe acceleration profile for the arm.

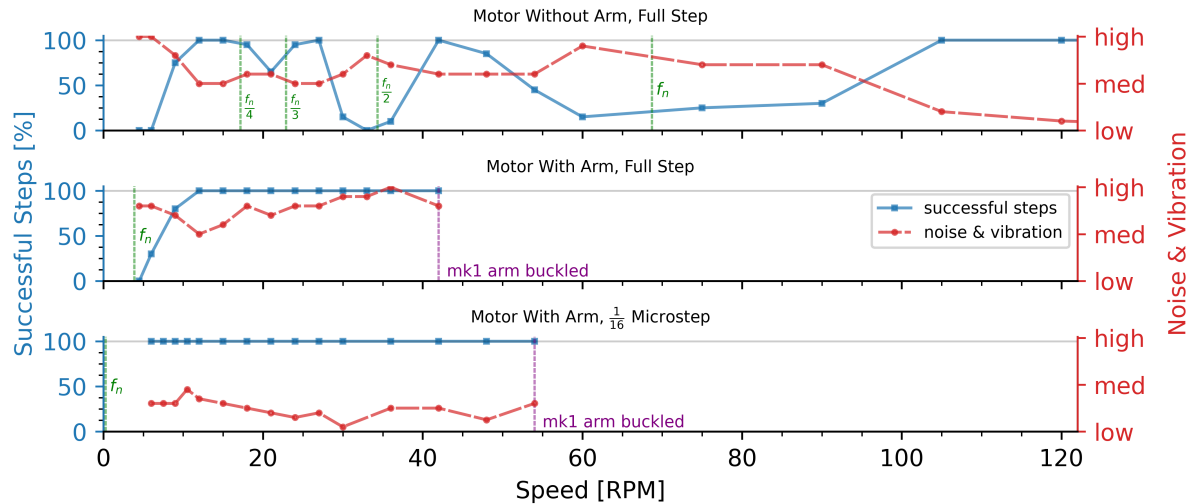


Figure 3: Noise, vibration and steps lost by motor C in different configurations. The natural frequencies estimated by calculation in section 2.1.2 are also shown.

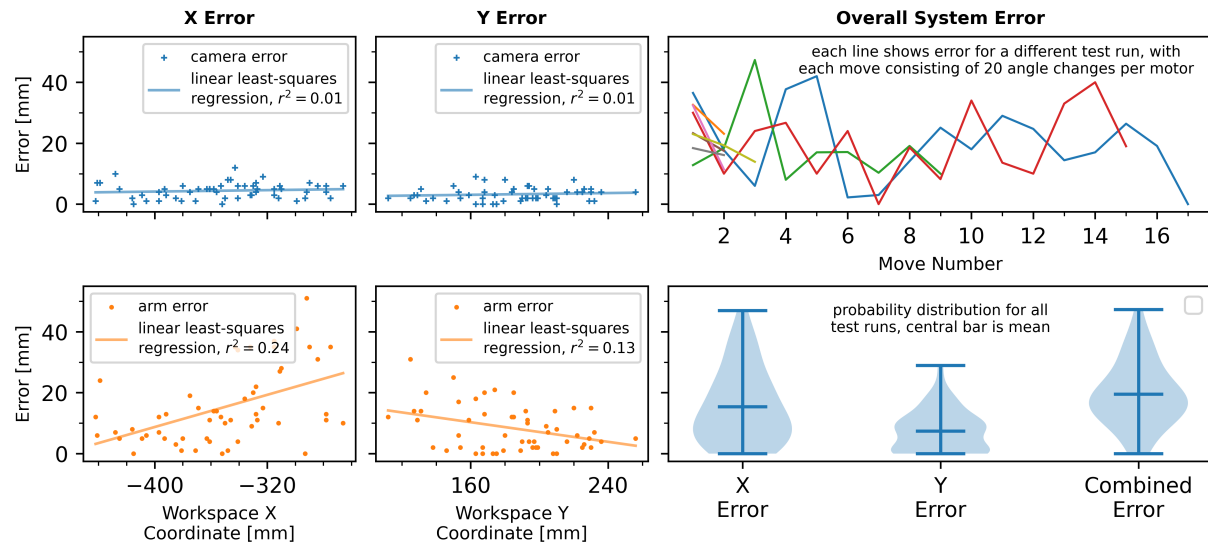


Figure 4: Results from testing the integrated system, including separate measurements taken for the error between the actual coordinate and the coordinate the camera measured, and the actual coordinate and the coordinate the arm moved to.

When testing the accuracy of the arm, the results (figure 4) showed a greater level of error on the right side of the workspace than the left, with the coefficient of determination for the regression lines showing a moderate correlation. The two most likely factors causing this are that the tests were done with the fixed bar lengths described previously, an arm configuration which is less optimal for manoeuvring in certain workspace areas, especially on the right-hand side of the workspace, and that the implementation of the kinematics algorithm used for testing did not select the arm mode to use for a move based on optimal precision. The primary contributors to arm error shown in figure 4 are arm configuration (mode and bar lengths), and the motor angle resolution of 1.8° . This error could be reduced by the arm being able to autonomously change its bar lengths to optimise for a particular workspace or movement (work which is in progress with a separate project), modifications to the kinematics algorithm, or by improving motor resolution either by using microstepping to increase resolution or by using servo motors. For stepper motors, position resolution does not increase linearly with step size reduction due to factors including

static position error, and torque at microstep positions is lower than at full step positions, so moving to a correctly specified servo motor is likely to be the more effective choice for reducing arm position error.

3.2 Communications

Routines sending different types and lengths of messages between the COM and the MCU in both directions were run (see S2.1 motorController.ino:debug_message_repeater), with the messages manually reviewed. Approximately 5% of messages during initial startup and handshake would fail to be correctly sent, possibly due to the high rate of two-way traffic, but it was found that the algorithms would detect the incorrectly formed message sentence, discard it, and usually resume communications successfully. During normal operation, when traffic follows a more predictable routine, no failures were observed. This confirms that the underlying error checking in the USB protocol, combined with the application layer message protocol, is sufficient for the present use case. Figure 5(a) shows that the calculated time for sending a 20-angle message agreed with the mean measured time (both 14.8 ms). Towards the end of testing, the typical size of angle messages was doubled from 10 angles to 20 angles because this was found to be beneficial for the movements generated by the kinematics algorithm. Due to this, it may be beneficial to swap to the faster Uno R4 MCU, and/or to swap to the binary communications algorithm to speed up message transmission times.

3.3 Computer Vision

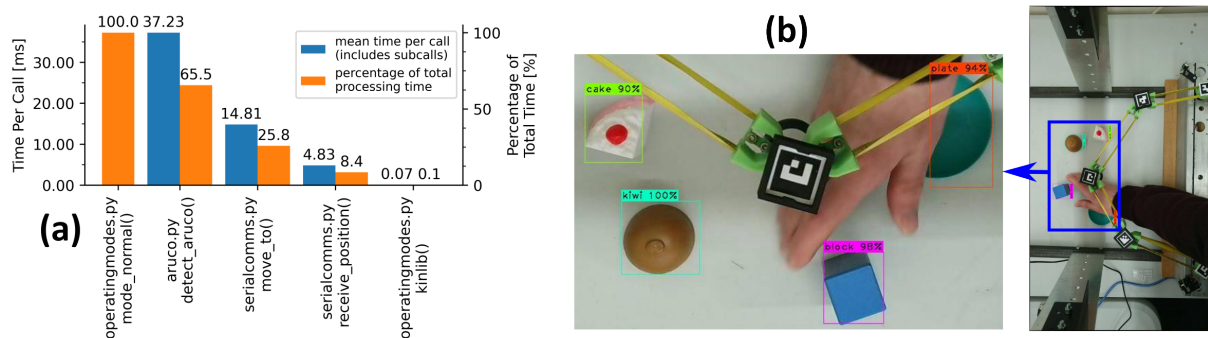


Figure 5: Testing the COM. **(a)** Processing time for the most resource-intensive algorithms on the Jetson Orin Nano (COM) when running the main routine, with Aruco marker detection run on every frame. **(b)** Screenshot of frame annotations showing detections and classifications from the YOLO CNN.

Figure 4 shows the results from testing the computer vision system by measuring different coordinates in the workspace. There was no correlation between error and location in the workspace, indicating that camera calibration to remove image distortion worked well. When tested using the distance calibration image several times, the repeatability of measurements was found to be in the range of tenths of a millimetre. The primary cause for the reduced accuracy seen in figure 4 is likely the camera not being perfectly perpendicular to the workspace, which was done visually. Approaches to reduce this error could include an improved method of aligning the camera or using two cameras in binocular vision, although this would increase cost and be more computationally intensive.

The detection algorithm was able to track the Aruco markers on every frame of video where the marker was not occluded, with measurements taken using the markers on the motors found to be repeatable to less than 1 mm. Figure 5(b) shows a detection by the YOLO CNN run on the COM. To evaluate the accuracy of the CNN, 60 frames of video were manually reviewed. The frames had between 0 and 4 objects each, with the majority having 3 objects (20 frames) or 4 objects (32 frames). Per frame, the mean time to resize the image from 1920×1080 to 480×256 was 2.0 ms, and the mean time for the

CNN to detect and classify the objects was 25.5 ms. There was high variability in the time for detection and classification, ranging from 6.5 ms to 34.1 ms, with a standard deviation of 10.4 ms. This confirms that the CNN can run at approximately 36 frames per second, possibly faster with further optimisations. Optimisations could include sizing the camera sensor and CNN so that image resizing is not required, and/or “bag of freebies” training optimisations that are possible with this particular CNN [4].

Provided that the post-processing algorithm selects the highest scoring classification for co-located boxes, and using a detection threshold of 50%, the CNN correctly detected the objects 99.6% of the time, and correctly classified them 92.2% of the time, with the classification errors all being confusion of the plate with the block when partially occluded and in a shadow. Further training is likely to improve this. This demonstrates the capability of the COM to use YOLO CNN based object detection and classification, opening up the possibility of more complex computer vision tasks for the system.

3.4 Integrated System

Figure 4 shows a comparison between the camera, arm and overall system error. The mean camera error was 5.9 mm (standard deviation 2.5 mm), the mean arm error 19.7 mm (standard deviation 11.0 mm), and the mean overall system error 19.5 mm (standard deviation 10.7 mm); overall system error mean is slightly lower than arm error because camera error must have been in the direction of correcting arm movement by an overall greater amount than it exacerbated it by. This level of precision makes the system suitable for lower precision tasks, such as manipulating the camera on an inspection robot or pick and place of easy to move parts. The system would also be suitable for playing air hockey (as is shown in figure 1), however this wasn't able to be tested because the algorithm to hit the puck would require the COM knowing the viable workspace of the arm in its present configuration, something that the kinematics library is not currently able to provide.

An analysis of algorithm execution timings on the COM is shown in figure 5(a). These were measured over a period of 15 minutes while running the main routine with Aruco marker detection on every frame. As can be seen, the marker detection took both the longest average time per call of the function and used the most processing time overall, with USB communication being the next most resource intensive task. The kinematics shared library was found to execute very efficiently. The YOLO CNN executed quicker than the Aruco detection algorithm (27.5 ms as opposed to 37.2 ms). This gives an effective average main loop rate for the overall system of 18 passes per second when running Aruco detection on every frame, or 21 passes per second when running the CNN on every frame. Increasing the speed of inference by the CNN, as discussed in section 3.3, combined with increasing the speed of communicating messages, as discussed in section 3.2, should be able to achieve an overall rate in excess of 30 passes per second, allowing the system to execute tasks that require relatively quick real-time responses. To accomplish tasks with simpler detection requirements, such as air hockey, where the puck has a shape and colour distinctive from the rest of an image frame, a simpler HSV (hue, saturation, value) colorspace based detection algorithm could be used, which would be far less computationally intensive [27].

The system (excluding the COM) had an average current draw of 0.79 A at a supply voltage of 24 V. The current draw by the COM plus peripherals, including allowance for conversion from 24 V to 5 V, was estimated as 0.5 A when running the YOLO CNN constantly. Assuming the arm was moving for 75% of its operational time, the total system would draw approximately 1.3 A, allowing over 3 h of operation if powered by an inexpensive 0.7 kg 4 A · h cordless drill battery, which would be suitable for intermittent use such as by a person requiring an arm for mobility assistance. A mobile industrial system could be powered by larger quick change batteries and only require the battery changing once per shift.

The overall cost of the system was kept low; component costs were motors (each) \$25, fans (pair) \$7, camera \$36, MCU (for production, estimated) \$5, power supply (for production, estimated) \$30, driver circuit/limit switches/mechanical parts/wiring \$60. This brings the total cost for the arm system, excluding the gantry, motor mount and COM, to \$188, comparing favourably to the estimated \$380 cost

of the system designed by Shinjo et al. [27]. The price of the Nvidia Jetson Orin Nano is around \$250.

4 Conclusion

The prototype system produced by this project has met the objectives set for it. The system was able to be constructed for a low total cost, and could accomplish tasks such as moving the arm to an object detected by the camera, no matter the length of the arm bars. Some tasks, such as playing air hockey, were not able to be tested because at the time of testing, the kinematics library was not able to provide data on the viable workspace of the arm, meaning that algorithms could not quickly pick a movement that they knew the arm would be capable of. The mk1 arm was made from a thin tape spring, which was found to be problematic once it fatigued, so a mk2 arm using a thicker tape spring has been designed with input from this project.

It was identified that, for lower precision applications, open-loop stepper motors are a viable choice for this system and are significantly cheaper than servo motors. Torque and resonance issues are common challenges associated with stepper motors that were exacerbated by the tape spring of the arm, but it was demonstrated that these issues can be overcome by using microsteps (but only stopping at full step positions), and having motors with sufficient torque run at an appropriate stepping rate and with rigid motor mounts.

The monocular camera system was found to be adequate for measuring distances when the camera is able to be placed perpendicular to the workspace, secured firmly, and the distance to the objects it is measuring is known. There are some tasks, such as warehouse pick and place, where this system would perform adequately, however for more demanding or dynamic environments, a dual camera binocular vision system may be more robust against measurement errors. The Aruco fiducial markers were found to be an accurate solution to detecting and measuring the positions of the motors and arm joints, but for real-time tasks such as moving the arm in relation to an object, the YOLO CNN was found to be more versatile and faster to execute, although it needs to be trained correctly on the desired objects prior to being deployed.

The modular design of the architecture in this project allows easy changing of both software algorithms and hardware, as was shown by swapping the prototype to use different cameras, computers, control algorithms, kinematics algorithms and other modules easily and quickly during this project. This will allow for quick testing and iteration of the system during further research, and for a final product that can be reconfigured to adapt to specific tasks or use cases with relative ease. Future work may want to develop the cheaper version of the system presented in this paper alongside a higher-performance and more precise version which uses servo motors and a second camera, at a modest increase in system hardware costs.

Overall, it was found that with further development, this arm system could become a viable product. The theorised strengths of the system (cheap, compact and lightweight) were shown by this project to be practical strengths that materialised. This project also showed that a COM such as the Jetson Orin Nano complements the rest of the system well.

Acknowledgements

I would like to thank my supervisor, Dr Andres San Millan Rodriguez, who designed the initial arm prototype and guided me through this project, and Zayne Turnbull for writing the kinematics algorithm.

References

- [1] P. Acarnley. *Stepping Motors: A Guide to Theory and Practice*. The Institution of Engineering and Technology, 4th edition, 2002. doi: 10.1049/PBCE063E. URL <https://digital-library.theiet.org/doi/abs/10.1049/PBCE063E>.
- [2] M.-C. Arva, M. Stanica, and N. Anghel. Analysis On Vibration and Resonance Characteristics of An Low Speed 3-Phase Stepper Motor. In *2018 10th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)*, pages 1–5, 2018. doi: 10.1109/ECAI.2018.8678993.
- [3] K. Balakrishnan, B. Umamaheswari, and K. Latha. Identification of Resonance in Hybrid Stepper Motor Through Measured Current Dynamics in Online for Accurate Position Estimation and Control. *IEEE Transactions on Industrial Informatics*, 9(2):1056–1063, 2013. doi: 10.1109/TII.2012.2221722.
- [4] A. Bochkovskiy, C. Wang, and H. M. Liao. YOLOv4: Optimal Speed and Accuracy of Object Detection. *CoRR*, abs/2004.10934, 2020. URL <https://arxiv.org/abs/2004.10934>.
- [5] L. Campos, F. Bourbonnais, I. Bonev, and P. Bigras. Development of a Five-Bar Parallel Robot With Large Workspace. *Proceedings of the ASME Design Engineering Technical Conference*, 2, 08 2010. doi: 10.1115/DETC2010-28962.
- [6] Y.-H. Chang, C.-H. Liang, and C.-C. Lan. An End-Effector Wrist Module for the Kinematically Redundant Manipulation of Arm-Type Robots. *Mechanism and Machine Theory*, 155:104064, 2021. ISSN 0094-114X. doi: <https://doi.org/10.1016/j.mechmachtheory.2020.104064>. URL <https://www.sciencedirect.com/science/article/pii/S0094114X20302846>.
- [7] S. Charette. Darknet/YOLO Licence, 2023. URL <https://darknetcv.ai/api/License.html>.
- [8] L. S. Dalenogare, G. B. Benitez, N. F. Ayala, and A. G. Frank. The Expected Contribution of Industry 4.0 Technologies for Industrial Performance. *International Journal of Production Economics*, 204:383–394, 2018. ISSN 0925-5273. doi: <https://doi.org/10.1016/j.ijpe.2018.08.019>. URL <https://www.sciencedirect.com/science/article/pii/S0925527318303372>.
- [9] S. Dang, H. Huang, and X. Li. About the Parsing of NMEA–0183 Format Data Streams in GPS. In N. Xiong, M. Li, K. Li, Z. Xiao, L. Liao, and L. Wang, editors, *Advances in Natural Computation, Fuzzy Systems and Knowledge Discovery*, pages 1282–1289, Cham, 2023. Springer International Publishing. ISBN 978-3-031-20738-9.
- [10] T. Ding, B. Li, H. Liu, Y. Peng, and Y. Yang. Planar Multi-Closed-Loop Hyper-Redundant Manipulator Using Extendable Tape Springs: Design, Modeling, and Experiments. *IEEE Robotics and Automation Letters*, 7(3):6630–6637, 2022. doi: 10.1109/LRA.2022.3176063.
- [11] L. Erwin-Christian, C. Valentin, D. Tivadar, O. Alexandru, T. Elida-Gabriela, and S. Melania-Olivia. Optimal Synthesis of Five-Bar Linkage Based on Singularity-Free Workspaces with Pre-defined Shapes. *Robotics*, 13(12), 2024. ISSN 2218-6581. doi: 10.3390/robotics13120173. URL <https://www.mdpi.com/2218-6581/13/12/173>.
- [12] S. Garrido-Jurado, R. Muñoz-Salinas, F. Madrid-Cuevas, and M. Marín-Jiménez. Automatic Generation and Detection of Highly Reliable Fiducial Markers Under Occlusion. *Pattern Recognition*, 47(6):2280–2292, 2014. ISSN 0031-3203. doi: <https://doi.org/10.1016/j.patcog.2014.01.005>. URL <https://www.sciencedirect.com/science/article/pii/S0031320314000235>.

- [13] F. L. Guan, X. Y. Wu, and Y. W. Wang. The Mechanical Behavior of the Double Piece of Tape Spring. In D.-S. Huang, M. McGinnity, L. Heutte, and X.-P. Zhang, editors, *Advanced Intelligent Computing Theories and Applications*, pages 102–110, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg. ISBN 978-3-642-14831-6.
- [14] G. He, C. Sparks, and N. Gravish. Grasping and Rolling In-Plane Manipulation Using Deployable Tape Spring Appendages. *Science Advances*, 11(15):eadt5905, 2025. doi: 10.1126/sciadv.adt5905. URL <https://www.science.org/doi/abs/10.1126/sciadv.adt5905>.
- [15] A. Hughes and B. Drury. *Electric Motors and Drives - Fundamentals, Types and Applications*. Newnes, Elsevier, 5th edition, 2019.
- [16] T. Igoe. *Making Things Talk*. O'Reilly, 2007.
- [17] A. Joubair, M. Slamani, and I. A. Bonev. Kinematic Calibration of a Five-Bar Planar Parallel Robot Using All Working Modes. *Robotics and Computer-Integrated Manufacturing*, 29(4):15–25, 2013. ISSN 0736-5845. doi: <https://doi.org/10.1016/j.rcim.2012.10.002>. URL <https://www.sciencedirect.com/science/article/pii/S0736584512001214>.
- [18] H. Kagermann, W. Wahlster, and J. Helbig. *Recommendations for Implementing the Strategic Initiative INDUSTRIE 4.0 - Final Report of the Industrie 4.0 Working Group*. acatech National Academy of Science and Engineering, 2013.
- [19] C. Li, Z. Liu, L. Li, Z. Ji, C. Li, J. Liang, and Y. Li. Improved PPO Optimization for Robotic Arm Grasping Trajectory Planning and Real-Robot Migration. *Sensors*, 25(17), 2025. ISSN 1424-8220. doi: 10.3390/s25175253. URL <https://www.mdpi.com/1424-8220/25/17/5253>.
- [20] H. M. Y. C. Mallikarachchi and S. Pellegrino. Deployment Dynamics of Ultrathin Composite Booms with Tape-Spring Hinges. *Journal of Spacecraft and Rockets*, 51(2):604–613, 2014. doi: 10.2514/1.A32401. URL <https://doi.org/10.2514/1.A32401>.
- [21] A. A. Murat and M. S. Kiran. A Comprehensive Review On YOLO Versions for Object Detection. *Engineering Science and Technology, an International Journal*, 70:102161, 2025. ISSN 2215-0986. doi: <https://doi.org/10.1016/j.jestch.2025.102161>. URL <https://www.sciencedirect.com/science/article/pii/S2215098625002162>.
- [22] Ooznest. NEMA17-114-200 Specifications. URL <https://ooznest.co.uk/product/8-2kg-cm-114oz-in-2-00a-1-8-nema17-hybrid-bipolar-stepper-motor/>.
- [23] O. G. Osele, A. M. Okamura, and B. H. Do. A Lightweight, High-Extension, Planar 3-Degree-of-Freedom Manipulator Using Pinched Bistable Tapes. In *2022 International Conference on Robotics and Automation (ICRA)*, page 1190–1196. IEEE, May 2022. doi: 10.1109/icra46639.2022.9811976. URL <http://dx.doi.org/10.1109/ICRA46639.2022.9811976>.
- [24] J. Redmon, S. K. Divvala, R. B. Girshick, and A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. *CoRR*, abs/1506.02640, 2015. URL <http://arxiv.org/abs/1506.02640>.
- [25] RS Components. RS PRO Hybrid, Permanent Magnet Stepper Motor 535-0467 Datasheet. . URL <https://docs.rs-online.com/0330/A7000000008880659.pdf>.
- [26] RS Components. RS PRO Hybrid, Permanent Magnet Stepper Motor 892-8732 Datasheet. . URL <https://docs.rs-online.com/d369/A7000000008919642.pdf>.

- [27] M. Shinjo, C. C. Beltran-Hernandez, M. Hamaya, and K. Tanaka. Low-Cost Air Hockey Robot Using a Five-Bar Linkage Mechanism Driven by Position-Control Servomotors. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5978–5985, 2024. doi: 10.1109/IROS58592.2024.10801458.
- [28] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González. A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction*, 5(4):1680–1716, 2023. ISSN 2504-4990. doi: 10.3390/make5040083. URL <https://www.mdpi.com/2504-4990/5/4/83>.
- [29] Texas Instruments. DRV8825 Stepper Motor Controller IC Datasheet revision F. 2014. URL <https://www.ti.com/lit/ds/symlink/drv8825.pdf>.
- [30] Ultralytics Inc. Ultralytics Github Issue 7440 - Regarding Pricing, 2024. URL <https://github.com/orgs/ultralytics/discussions/7440>.
- [31] Ultralytics Inc. Ultralytics License, 2026. URL <https://www.ultralytics.com/license>.
- [32] Ömer Soykasap. Analysis of Tape Spring Hinges. *International Journal of Mechanical Sciences*, 49(7):853–860, 2007. ISSN 0020-7403. doi: <https://doi.org/10.1016/j.ijmecsci.2006.11.013>. URL <https://www.sciencedirect.com/science/article/pii/S0020740306002852>.